

The Pattern Everyone's Quietly Building: A Personal Wiki for Your AI

By Mercedes Perez Capilla

Over the past few weeks I've watched a quiet pattern emerge across my network. Practitioners I respect — engineers, operators, consultants, product leaders — are all converging on the same idea, often without knowing about each other. They're building personal wikis designed for AI to read.

Not knowledge bases for themselves to read. *For their AI to read.*

The shapes differ — Obsidian and Claude Code, Cowork, Notion with custom prompts, Roam, hand-rolled markdown folders — but the underlying pattern is the same. I built mine and it's already changed how I work. I want to share what I built, what others are building, and why I think this is worth your time.

The shared pattern

Strip away the tool choices and the pattern is:

1. A set of **structured context documents** describing who you are, what you work on, how you think, and what you care about.
2. A **persistent wiki** of interlinked markdown pages that your AI maintains as you feed it sources — articles, papers, meeting notes, conversations.
3. A **schema file** (often called `CLAUDE.md`) that tells the AI the rules: how to structure pages, when to update vs. create, what to flag.
4. A daily habit of treating the wiki as the *output* of your AI sessions, not a side effect.

The result, as Anne Despain put it in her own LinkedIn post about the pattern: *an AI that stops giving generic advice and starts operating like someone who has worked with you for years.*

That's the whole point.

The thinkers I owe the shape to

Two people did the heavy intellectual lifting that my own setup is built on:

Andrej Karpathy published the *LLM Wiki* concept — a personal wiki designed specifically for AI consumption, structured so language models can build, maintain, and query it. His gist describes three operations: ingest a source, query the wiki, lint for contradictions. It's the architectural backbone.

Allie K Miller built the *AI Context Vault* — a set of interview prompts that produce layered personal context documents covering who you are, what you work on, how you think, and how you communicate. This is what makes your AI start a session knowing you instead of cold. She has since extended her own setup into a full wiki ("Claudeopedia") with custom skills layered on top — proof that the pattern keeps evolving in the hands of the people using it.

I built my Context Vault following Allie's approach in January 2026, and in May I built the wiki following Karpathy's pattern and Allie's Claudeopedia. The move that unlocked everything was feeding my existing Context Vault *into* the wiki as a source — so the wiki itself now compounds not just on external knowledge but on my own evolving context. That combination is what I want to share.

The variations I've seen

Once you start looking, you see the pattern everywhere. A few that have stuck in my head:

The Cowork build (Helen Lee Kupp showed how to do this in minutes on LinkedIn). Fastest path in. You stay in a friendly UI and get a working wiki without setup overhead. Strong for people who want the value of the pattern without living in a terminal.

The Obsidian + Claude Code build (what I did). A folder of markdown files. Claude Code does the writing and maintenance. Obsidian provides the view — graph view, backlinks, search. Pure local files, no database, no vendor lock-in. Strong if you want full control and a beautiful visual layer.

The Notion build. Same pattern, but lives in Notion's database model. Strong if your team already lives there and you want easier sharing.

The hand-rolled MCP build. Engineers piping data sources through Model Context Protocol connectors directly into the wiki. Strong if you have specific data sources (Slack, email, calendar) you want ingested automatically.

The point isn't which tool is best. The point is the pattern works across all of them.

If you're not sure where to start, go with Claude Code and a markdown wiki. It's the most direct expression of the pattern, gives you full control, and everything else builds on the same foundation.

A note on NotebookLM. Several people have asked me about it. NotebookLM is excellent for what it does — one-off deep dives into a fixed set of documents, with audio overviews and grounded Q&A — but it isn't really suited to this pattern. It retrieves; it doesn't compound. There's no persistent editable artefact the AI maintains, no cross-references between pages, no way for new sources to update existing knowledge. If you want a tool for interrogating a specific document set, it's great. If you want a knowledge base that gets smarter the longer you use it, look elsewhere.

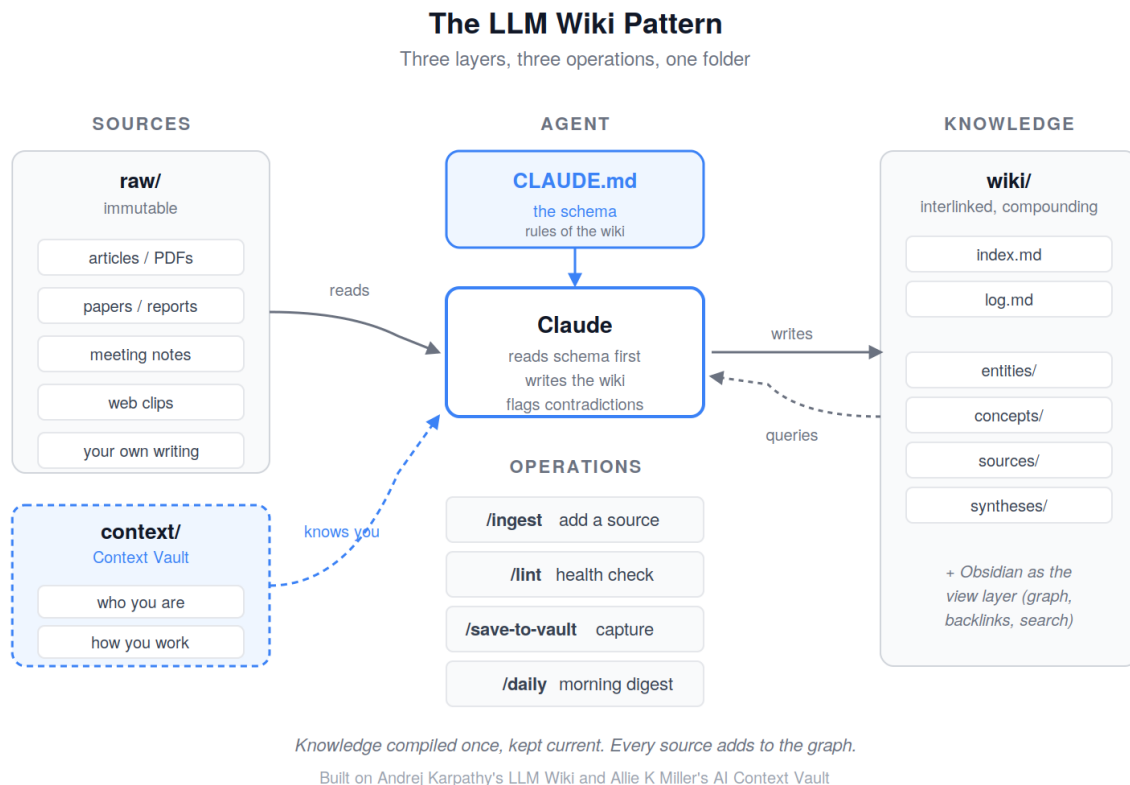
What mine looks like

One folder. Three layers:

```
my-wiki/
├── CLAUDE.md      ← The schema. Rules of my wiki.
├── context/       ← Layered context documents. Who I am, how I work.
├── raw/           ← Source documents. AI reads, never modifies.
└── wiki/          ← AI owns this. Interlinked markdown pages.
    ├── index.md   ← Every page with a one-line summary.
    └── log.md      ← Append-only record of what's happened.
```

Obsidian sits on top of the same folder as the view layer. The graph view is what makes this tangible — every `[[wikilink]]` Claude creates becomes a visible connection. As the wiki grows, the graph reveals natural clusters: which concepts are central, which are isolated, where the gaps are. You stop reading individual pages and start seeing the shape of what you know.

Here's how the whole pattern fits together:



The LLM Wiki Pattern — three layers, three operations, one folder

Three operations, all run through Claude Code:

- **Ingest** — drop a source into `raw/`, Claude reads it, updates the wiki, cross-references existing pages, flags contradictions.
- **Query** — ask questions against the wiki. Claude searches relevant pages and synthesises an answer. Good answers get filed back into the wiki.
- **Lint** — periodically health-check the wiki for orphan pages, stale claims, missing cross-references.

That's it. No database. No embeddings. No subscription beyond Claude itself.

Extending the pattern

Once the basic wiki is running, the next move is automating the operations you find yourself repeating. I've built four slash commands that live alongside the wiki:

- **/ingest** — adds a new source, updates relevant pages, cross-references, and flags contradictions instead of silently overwriting.

- `/lint` — health-checks the wiki for contradictions, orphans, broken links, stale claims, and missing cross-references.
- `/save-to-vault` — captures decisions, insights, and open threads from a session before closing it.
- `/daily` — produces a read-only morning digest of what changed, what's open, and one suggested action.

This is where Allie's Claudeopedia work is most interesting — she's pushed the same idea further with custom skills for screenshot ingestion, time-range search, and a scheduled job that questions your assumptions against your recent writing. The pattern keeps inviting new extensions.

I'll go deeper on the commands and what I'm building next in Part 2.

Why I think it's worth your time

What's different about this pattern is that knowledge *compounds*. Every source you ingest makes the whole wiki smarter, not just bigger. Every question you ask leaves an artefact behind. Every week your AI knows you better.

A small example: I recently asked my wiki a question about a work problem and it pulled together context from three sources I'd ingested weeks apart — the kind of synthesis I'd previously have spent an hour doing by hand. That's the compounding return.

It's also genuinely accessible. You don't need to be a developer. If you can write a folder of text files, you can do this. Helen's Cowork approach gets you running in minutes. The investment is small. The compounding return, in my experience so far, is large.

Where to start

Pick the variation that matches where you already work:

- Already in Notion? Build it there. Anne Despain has written about this.
- Want the fastest path with no setup? Try Cowork. Helen Lee Kupp's post is the place to start.
- Want full control, local files, and a graph view? Obsidian + Claude Code, the route I took.

Whatever you choose, the three foundational reads are:

- Karpathy's LLM Wiki gist: gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
 - Allie K Miller's AI Context Vault: alliekmiller.com/ai-context-vault
 - Helen Lee Kupp's Cowork walkthrough (search her LinkedIn)
-

Coming next

This is Part 1. In Part 2 I'd like to compare notes properly — interview a few people in the RepresentAI community who are building variations of this and share what they've learned. If you've built one and would be willing to talk about it, find me on LinkedIn or in the RepresentAI Financial Services group.

In Part 3 I want to tackle the harder question I'm sitting with at work: *what would it take to bring this pattern into a regulated organisation, safely and at scale?* That one I'm still working on.

Mercedes Perez Capilla is a Product Lead for Transformation and AI Adoption in financial services. She is a Claude Certified Architect candidate and part of the Financial Services group at RepresentAI.